
Module 22 — Kubernetes

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Desired State	2
2.2 Requests and Limits	2
2.3 Kustomize	3
3. Step-by-Step Examples	3
3.1 Image and Deployment Script	3
3.2 The Base: Configuration and Secrets	4
3.3 The Deployment	4
3.4 Overlays: dev and prod	7
3.5 PostgreSQL as a StatefulSet	8
3.6 Autoscaling, Rolling Update and Rollback	9
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Run a local Kubernetes cluster with kind and load your own image into it
- Describe an application with a Deployment, a Service, a ConfigMap and a Secret
- Connect liveness, readiness and startup probes to Spring Boot Actuator
- Set resource requests and limits, and explain what each one does
- Build environment variants with Kustomize (base + overlays)
- Scale with a HorizontalPodAutoscaler, update with a rolling update, and roll back a broken version

Prerequisites: Module 20 (image, probes, graceful shutdown) · **Estimated time:** 5 hours
· **Docker, kind and kubectl required**

2. Concepts

2.1 Desired State

With Kubernetes you do not run commands like “start two containers”. You describe the **desired state** in YAML (“two replicas of this image, with these probes”) and apply it. Controllers compare the desired state with reality all the time and act: a crashed pod is replaced, a new image version is rolled out step by step.

Object	Purpose
Pod	one or more containers that run together; replaceable
Deployment	keeps N identical pods running, and updates them
Service	one stable name and address for all ready pods (load balancing)
ConfigMap / Secret	configuration and credentials, as environment variables or files
StatefulSet	pods with a stable identity and their own storage (databases)
HorizontalPodAutoscaler	changes the number of replicas by load

2.2 Requests and Limits

	Meaning	Effect
requests	what the pod gets for sure	the scheduler places pods by requests; the HPA computes its percentages from them
limits	the upper bound	memory above the limit: the container is killed (OOMKilled); CPU above the limit: throttled

The lesson sets a memory limit, but no CPU limit. A CPU limit slows the JVM down exactly when it needs CPU (startup, garbage collection), without protecting anything.

2.3 Kustomize

Kustomize (built into `kubectl`) combines a **base** with **overlays**. The base has what all environments share. An overlay changes only what differs (replicas, resources, configuration). No templates and no variables: plain YAML plus patches.

3. Step-by-Step Examples

Create the cluster once. The script also installs the metrics-server that the autoscaler needs:

```
scripts/kind-up.sh
```

3.1 Image and Deployment Script

The application is the one from module 20 (API, probes, graceful shutdown), plus a CPU load endpoint. The image is built with the same multi-stage Dockerfile. `deploy.sh` does all the steps:

```
modules/22-kubernetes/deploy.sh dev
```

1. `./mvnw package and docker build`
2. `kind load docker-image`: the kind node has its own image store and cannot see the images of your Docker. The Deployment therefore uses `imagePullPolicy: IfNotPresent`.

3. `kubectl apply -k modules/22-kubernetes/k8s/overlays/dev`
4. `kubectl rollout status`: waits until all pods are ready
5. A smoke test through the Service (port-forward)

```
deployment "bookstore" successfully rolled out
NAME                                READY   STATUS    RESTARTS   AGE
bookstore-5b6b97d4d7-wdkxg         1/1     Running   0           4s
postgres-0                          1/1     Running   0           82s
{"host": "bookstore-5b6b97d4d7-wdkxg", "name": "Bookstore (dev)", "currency": "TRY"}
```

3.2 The Base: Configuration and Secrets

```
configMapGenerator:
  - name: bookstore-config           # the real name gets a hash suffix, e.g.
    ↪ bookstore-config-7k2m9f4b8c
    literals:
      - BOOKSTORE_SHOP_NAME=Bookstore on Kubernetes
      - BOOKSTORE_SHOP_CURRENCY=TRY
secretGenerator:
  - name: bookstore-db
    literals:
      - SPRING_DATASOURCE_USERNAME=bookstore
      - SPRING_DATASOURCE_PASSWORD=bookstore # a dev value only – a real secret comes
    ↪ from outside the repository
```

A **generator** creates the ConfigMap with a hash of its content in the name (bookstore-config-tk8m75d8kc), and Kustomize writes this name into the Deployment. When a value changes, the name changes. The Deployment changes too, and Kubernetes rolls the pods. A hand-written ConfigMap would change silently: environment variables are read only when a container starts.

! Warning

A Secret is only base64-encoded, not encrypted. The password in the generator is a development value. In a real system, secrets come from outside the repository (a secret store, Sealed Secrets, External Secrets).

3.3 The Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

  name: bookstore
spec:
  replicas: 2
  selector:
    matchLabels:
      app: bookstore
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0 # never fewer ready pods than wanted during an
↪ update
      maxSurge: 1 # start one new pod at a time
  template:
    metadata:
      labels:
        app: bookstore
    spec:
      terminationGracePeriodSeconds: 30 # longer than
↪ spring.lifecycle.timeout-per-shutdown-phase (20 s)
      initContainers: # runs to completion before the application
↪ starts
        - name: wait-for-postgres
          image: pgvector/pgvector:0.8.6-pg18
          command: ["sh", "-c", "until pg_isready -h postgres -U bookstore -d bookstore;
↪ do sleep 2; done"]
        containers:
          - name: bookstore
            image: springbootedu/kubernetes:1.0.0
            imagePullPolicy: IfNotPresent # the image is loaded into kind, not pulled
↪ from a registry
            ports:
              - name: http
                containerPort: 8080
            envFrom:
              - configMapRef:
                  name: bookstore-config
              - secretRef:
                  name: bookstore-db
            env:
              - name: SPRING_DATASOURCE_URL
                value: jdbc:postgresql://postgres:5432/bookstore
            resources:
              requests: # what the scheduler reserves (and what the HPA
↪ percentages refer to)
                cpu: 250m
                memory: 384Mi
              limits:

```

```

        memory: 512Mi                # above this the container is killed
↪ (OOMKilled); no CPU limit: no throttling
        startupProbe:                # until it succeeds, the other probes wait
↪ (slow first start)
        httpGet:
            path: /actuator/health/liveness
            port: http
        periodSeconds: 2
        failureThreshold: 60         # up to 2 minutes to start
        livenessProbe:               # fails → the container is restarted
        httpGet:
            path: /actuator/health/liveness
            port: http
        periodSeconds: 10
        failureThreshold: 3
        readinessProbe:              # fails → the pod gets no traffic from the Service
        httpGet:
            path: /actuator/health/readiness
            port: http
        periodSeconds: 5
        failureThreshold: 2

```

- The **initContainer** waits until PostgreSQL answers. Without it, the first start fails (Flyway cannot connect), and Kubernetes restarts the pod a few times until the database is up.
- `envFrom` turns every key of the ConfigMap and the Secret into an environment variable. Boot's relaxed binding does the rest (module 20).
- `terminationGracePeriodSeconds: 30` is longer than Spring's `timeout-per-shutdown-phase: 20s`. Kubernetes sends `SIGTERM`, waits, and only then kills.

The probes point to the Actuator groups:

```

startupProbe:                        # until it succeeds, the other probes wait (slow first
↪ start)
    httpGet:
        path: /actuator/health/liveness
        port: http
    periodSeconds: 2
    failureThreshold: 60             # up to 2 minutes to start
livenessProbe:                      # fails → the container is restarted
    httpGet:
        path: /actuator/health/liveness
        port: http
    periodSeconds: 10
    failureThreshold: 3
readinessProbe:                    # fails → the pod gets no traffic from the Service
    httpGet:

```

```

    path: /actuator/health/readiness
    port: http
    periodSeconds: 5
    failureThreshold: 2

```

The **startup probe** protects a slow first start: until it succeeds, liveness does not run, so a JVM that needs 40 seconds is not restarted after 30.

ManifestPolicyTest checks such rules in every build, without a cluster. For example, Kubernetes must wait longer than the application's graceful shutdown:

```

@Test
void kubernetesWaitsLongerThanTheGracefulShutdown() throws IOException {
    int gracePeriod = (Integer) path(deployment(), "spec", "template", "spec",
        ↪ "terminationGracePeriodSeconds");
    String shutdownTimeout =
        ↪ Files.readString(Path.of("src/main/resources/application.yaml"))
            .lines().filter(line ->
                ↪ line.contains("timeout-per-shutdown-phase")).findFirst().orElseThrow();
    int springSeconds = Integer.parseInt(shutdownTimeout.replaceAll(".*: (\\d+)s.*",
        ↪ "$1"));

    assertThat(gracePeriod).isGreaterThan(springSeconds);
}

```

3.4 Overlays: dev and prod

k8s/	
base/	namespace, postgres, deployment, service, generators
overlays/dev/	1 replica, shop name "Bookstore (dev)"
overlays/prod/	more memory, HorizontalPodAutoscaler 2–5 replicas

```

apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: bookstore                                # also for the resources this overlay adds
↪ (hpa.yaml)
resources:
- ../../base
- hpa.yaml
patches:
- path: resources.yaml
configMapGenerator:
- name: bookstore-config

```

```
behavior: merge
literals:
  - BOOKSTORE_SHOP_NAME=Bookstore
```

`kubectl kustomize modules/22-kubernetes/k8s/overlays/prod` prints the result without applying it.

!! Caution

`namespace: bookstore` in the base applies only to the resources of the base. A resource that an overlay adds (here `hpa.yaml`) ends up in the default namespace, where it finds no Deployment. Every overlay therefore sets `namespace: too`, and Manifest-`PolicyTest` checks it. (This happened while writing this lesson.)

3.5 PostgreSQL as a StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: pgvector/pgvector:0.8.6-pg18
          env:
            - name: POSTGRES_DB
              value: bookstore
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: bookstore-db
                  key: SPRING_DATASOURCE_USERNAME
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
```

```

        name: bookstore-db
        key: SPRING_DATASOURCE_PASSWORD
      - name: PGDATA
        value: /var/lib/postgresql/data/pgdata
    ports:
      - containerPort: 5432
    readinessProbe:
      exec:
        command: ["pg_isready", "-U", "bookstore", "-d", "bookstore"]
        periodSeconds: 5
    volumeMounts:
      - name: data
        mountPath: /var/lib/postgresql/data
    volumeClaimTemplates:
      # every pod of the StatefulSet gets its own
      ↪ persistent volume
      - metadata:
          name: data
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 1Gi

```

A StatefulSet gives each pod a stable name (postgres-0) and its own volume from volumeClaimTemplates. The volume survives a restart of the pod. This is enough to learn with. In production, use a managed database or an operator (backups, replication, upgrades).

3.6 Autoscaling, Rolling Update and Rollback

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bookstore
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bookstore
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu

```

```
target:
  type: Utilization
  averageUtilization: 60
```

With load from a second pod (/api/work keeps the CPU busy), the autoscaler reacted within a minute:

```
modules/22-kubernetes/deploy.sh prod
kubectl -n bookstore run load --image=busybox:1.37 --restart=Never -- /bin/sh -c \
  'while true; do for i in 1 2 3 4 5 6 7 8; do wget -q -O-
    ↪ "http://bookstore/api/work?millis=200" > /dev/null & done; wait; done'
kubectl -n bookstore get hpa bookstore --watch          # replicas 2 → 4 → 5
kubectl -n bookstore delete pod load
```

After the load stops, the HPA waits 5 minutes (the default stabilization window) before it removes pods again.

A broken release and a rollback:

```
kubectl -n bookstore set image deployment/bookstore
↪ bookstore=springbootedu/kubernetes:1.0.1-broken
kubectl -n bookstore get pods          # the new pod: ErrImagePull – the 5 old pods
↪ still serve
kubectl -n bookstore rollout history deployment/bookstore
kubectl -n bookstore rollout undo deployment/bookstore
```

Because of `maxUnavailable: 0`, Kubernetes removes an old pod only when a new one is ready. A version that never becomes ready (wrong image, failing readiness) therefore never takes traffic away. The rollout just stops until you roll back.

Remove the cluster when you are done: `scripts/kind-down.sh`.

4. Common Mistakes and Best Practices

!! Caution

Liveness must not depend on the database or other services (module 20). On Kubernetes, a failing liveness probe restarts the pod, and a restart loop of all pods turns a database hiccup into an outage.

- **Do:** set memory requests and limits, and CPU requests. Without CPU requests, the HPA cannot compute a percentage.
- **Don't:** use `latest` as the image tag. Kubernetes cannot tell versions apart, and a rollback has nothing to return to.

- **Do:** give the JVM time to start (startup probe), and give graceful shutdown time to finish (terminationGracePeriodSeconds).
- **Don't:** change a hand-written ConfigMap and expect running pods to see it. Use a generator, or restart the Deployment (`kubectl rollout restart`).
- **Do:** check manifests in the build (a policy test, `kubectl kustomize`, or tools like `kubeconform`), not only in the cluster.

5. Summary

- Kubernetes works with desired state: Deployments keep pods running, Services give them one address.
- Probes: startup for the first start, liveness for “restart me”, readiness for “send me traffic”. All three come from Actuator.
- Requests are for scheduling and autoscaling, limits are upper bounds. Memory limit yes, CPU limit rather not.
- Kustomize builds environments from one base. Generators turn configuration changes into rolling updates.
- The HPA scales on CPU. `maxUnavailable: 0` plus readiness makes a rolling update safe, and `rollout undo` goes back.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Deploying to Kubernetes](#)
- [Kubernetes — Deployments · Probes](#)
- [Kubernetes — Resource management · HorizontalPodAutoscaler](#)
- [Kustomize · kind](#)